

Module 7: Advanced Verification Concepts - Workshop Companion

This workshop contains three labs designed to give you hands-on experience with the core pillars of modern functional verification: **Functional Coverage**, **SystemVerilog Assertions (SVA)**, **Advanced Stimulus Generation** and **Controlling Randomization Order**.

Lab 1: Measuring Stimulus Quality with Functional Coverage

Objective: Implement a covergroup to measure how well a randomized testbench is exercising the input space of a memory transaction class. You will define coverpoints for key properties, add bins for interesting value ranges, and create a cross to check for important combinations.

01

Create Transaction Class

Build the `mem_trans` class with randomized properties

02

Define Coverage Collector

Implement covergroup to sample transactions

03

Integrate and Analyze

Connect coverage to testbench and measure results

Part A: The Transaction Class and Testbench

The Goal: Create a basic class to model a memory transaction and a testbench to generate random instances of it. This provides the stimulus we will measure.

1. Create the file mem_trans.sv:

- Define a class named mem_trans.
- Add the following rand properties:
 - logic [7:0] m_addr;
 - logic [31:0] m_data;
 - bit m_is_write;
 - int m_burst_len;
- Add a constraint named burst_c to keep m_burst_len between 1 and 8 (inclusive).
- Add a simple display method to print the transaction's properties.

2. Create the testbench file tb_coverage.sv:

- Create a module named tb_coverage.
- In an initial block:
 - Declare a handle for mem_trans, let's call it txn.
 - Create a repeat (100) loop.
 - Inside the loop, create a new mem_trans object, randomize it successfully, and call its display method.

Part B: The Coverage Collector

The Goal: Define a covergroup to sample the transactions generated by the testbench.

1. Create the file coverage_collector.sv:

- Define a class named coverage_collector.
- It should have a constructor new() that takes no arguments.
- Define a covergroup named mem_trans_cg inside the class. This covergroup should be triggered by a formal argument of type mem_trans. For example: covergroup mem_trans_cg(mem_trans t);

Inside the covergroup:

Address Coverpoint

Define a coverpoint for the `t.m_addr` property. Create bins for different ranges: low (0-63), mid (64-191), and high (192-255).

Write Operation Coverpoint

Define a coverpoint for the `t.m_is_write` property. SystemVerilog will create automatic bins for 0 and 1.

Burst Length Coverpoint

Define a coverpoint for the `t.m_burst_len` property. Create bins for a single transfer (`single = {1}`) and burst transfers (`burst = {[2:8]}`).

Cross Coverage

Define a cross between `is_write` and `burst_len`. This will check if you have performed both single and burst transfers for both reads and writes.

Add a method to the class function `void sample(mem_trans t);` which calls the covergroup's sample method: `mem_trans_cg.sample(t);`

Part C: Integration and Analysis

The Goal: Integrate the coverage collector into the testbench and analyze the results.

1. Update the testbench file `tb_coverage.sv`:

- Include both `mem_trans.sv` and `coverage_collector.sv`.
- In the initial block, before the loop, create an instance of your `coverage_collector` class.
- Inside the repeat loop, after successfully randomizing and displaying the transaction, call the `sample()` method of your collector object, passing the `txn` handle to it.
- After the loop finishes, print the overall coverage percentage using the built-in `$get_coverage()` system function. For example:
`$display("Final Coverage = %0.2f%%", $get_coverage());`
- Run the simulation and observe the final coverage report.

 **Professional Context: Coverage in Regressions** In this lab, we use `$get_coverage()` for instant feedback in a single test. In a professional environment, you would never rely on this for signing off on a project. The standard industry practice is to run hundreds or thousands of tests in an automated "regression." Each test run generates a coverage database file (.ucdb file in QuestaSim). After the regression completes, a command-line tool is used to **merge** all these individual databases into a single, master database. This merged view gives you the true, cumulative coverage across your entire verification effort.

Lab 2: Verifying a DUT with SystemVerilog Assertions

Objective: Write concurrent assertions to verify the rules of a simple, fixed-priority bus arbiter. You will check for mutual exclusion, grant logic, and liveness.

Part A: The Design Under Test (DUT)

The Goal: Understand the behavior of the simple arbiter you will be verifying.

1. Create the file arbiter.sv:

- Create a module named arbiter.
- **Ports:** input logic clk, reset, input logic req_0, req_1, output logic gnt_0, gnt_1.
- **Logic:** Implement a fixed-priority arbiter.
 - On the positive edge of clk, if reset is active, all grants should be low.
 - Otherwise, if req_0 is high, gnt_0 should be high and gnt_1 should be low.
 - Otherwise, if req_1 is high, gnt_1 should be high and gnt_0 low.
 - If neither request is active, both grants should be low.

Part B: The SVA Checker Module

The Goal: Write formal properties that capture the design intent of the arbiter.

1. Create the file `arbiter_checker.sv`:

- Create a module named `arbiter_checker` with the same ports as the arbiter DUT.
- All assertions should be specified to be checked at the positive edge of the clock.

Assertion 1 (Mutual Exclusion)

The most critical rule is that both grants can never be active at the same time. Write an assert property to check that `!(gnt_0 && gnt_1)` is always true. Give it a meaningful name like `A1_MUTUAL_EXCLUSION`.

Assertion 2 (Grant Implies Request)

A grant should only be given if it was requested. Write two assert property statements: One to check that if `gnt_0` is high, `req_0` must also have been high in the previous cycle. Use the implication operator `|->`. A second one for `gnt_1` and `req_1`.

Assertion 3 (Cover Liveness)

We want to ensure that our testbench actually creates the condition where a request leads to a grant. Write a cover property to check that if `req_0` is asserted, `gnt_0` is asserted in the same clock cycle.

Part C: The Testbench

The Goal: Instantiate the DUT and checker, drive random stimulus, and observe the assertion results.

1. Create the file `tb_arbiter.sv`:

- Create a top-level module named `tb_arbiter`.
- Instantiate the arbiter DUT and the `arbiter_checker`. Connect them to the same signals.
- Generate a clock signal.
- In an initial block:
 - Toggle the reset.
 - Write a loop to drive the `req_0` and `req_1` signals. Use `$urandom_range` to randomly assert them for a few cycles to create interesting scenarios.
 - Run the simulation long enough to test various request combinations. Observe if any assertions fail.

Lab 3: Advanced Constrained Randomization

Objective: Use advanced constraint features (dist, solve...before, and foreach) to create weighted and relational stimulus scenarios. This lab builds upon the mem_trans class from Lab 1.

Part A: The Scenario-Based Transaction Class

The Goal: Modify the memory transaction class to support more complex, weighted scenarios.

1. Create a new transaction file scenario_trans.sv (you can copy from mem_trans.sv).
2. Modify the scenario_trans class:
 - **Add an Opcode:** Create an enum for different operation types: typedef enum {WRITE, READ, IDLE} opcode_e;. Add a rand opcode_e opcode; property to the class. Remove the old is_write property.
 - **Weighted Opcodes:** Add a new constraint using the dist operator to control the distribution of opcodes. Make WRITE and READ operations each have a weight of 40, and IDLE a weight of 20.

Relational Constraint

Add a constraint block to enforce a rule: IDLE operations should only ever be single-cycle transfers (i.e., if opcode is IDLE, then burst_len must be 1). Use the `->` implication operator inside the constraint.

Update the `display()` method to print the new opcode enum value instead of `is_write`.

Part B: The Scenario Testbench

The Goal: Create a new testbench that generates and observes the behavior of the new scenario-based transaction class.

1. Create the testbench file `tb_scenario.sv`:

- Create a module named `tb_scenario`.
- include `"scenario_trans.sv"`.
- In an initial block:
 - Create a handle for the `scenario_trans` class.
 - Create a repeat (50) loop.
 - Inside the loop, create a new object, randomize it, and display its contents.

2. Run and Analyze:

1

Compile and Run

Compile and run the simulation.

2

Examine Output

Examine the output log. Over the 50 transactions, do you see roughly twice as many READ/WRITE operations as IDLE operations?

3

Verify Constraints

Verify that whenever an IDLE transaction is printed, its burst length is always 1.

📌 **Key Learning:** This lab demonstrates how advanced SystemVerilog constraint features enable you to create realistic, weighted test scenarios that better reflect real-world usage patterns while maintaining strict design rules through relational constraints.

Handout 4: Lab 4 - Controlling Randomization Order

Objective: Learn two critical techniques for managing dependencies in randomization: using `solve...before` to control the order in which random variables are solved, and using `post_randomize()` to calculate values after randomization is complete.

Part A: Ordering with solve...before

The Goal: Create a transaction class for a variable-length data packet. The size of the packet's payload array must be determined *before* the solver attempts to randomize the contents of that array. This is a common scenario that requires ordering constraints.

01

Create the file `ordered_trans.sv`:

- Define a class named `ordered_trans`.
- Add the following properties:
 - `rand int m_length;`
 - `rand byte m_payload[]; // A dynamic array`

Add a constraint for length: Constrain `length` to be between 4 and 8 (inclusive).

Add a constraint for payload size: Add a constraint that the payload array's size must be equal to the length. Use `m_payload.size() == length`.

Add the ordering constraint: Use `solve m_length before m_payload;` to explicitly tell the constraint solver to determine a value for `m_length` first.

Add a display method to print the length and the contents of the payload array.

02

Create the testbench `tb_ordering.sv`:

- Include the `ordered_trans.sv` file.
- In an initial block, create and randomize an instance of the `ordered_trans` class a few times in a loop.
- Call the display method after each successful randomization.

Observe: Verify that the size of the printed payload array always matches the randomized length.

Part B: Self-Checking with post_randomize()

The Goal: Model a simplified IP packet where the header checksum is calculated procedurally *after* all other header fields have been randomized. The testbench will then be made **self-checking** by adding a verification method to the class itself.

01

Create the file ip_packet.sv:

- Define a class named ip_packet.
- Add the following properties:
 - rand bit [31:0] m_src_addr;
 - rand bit [31:0] m_dst_addr;
 - rand bit [7:0] m_protocol;
 - bit [15:0] m_header_checksum; // Note: NOT rand

Implement post_randomize():

- Create the function void post_randomize();.
- Inside the function, calculate a simple checksum (e.g., the sum of the other fields) and assign it to m_header_checksum.

Implement a self-checking function:

- Create a function function bit verify_checksum();.
- Inside this new function, recalculate the checksum from the current values of the random fields.
- Compare the newly calculated checksum with the one stored in the header_checksum property.
- If they match, return 1 (true). If they don't, print an \$error message and return 0 (false).

Add a display method to print all fields.

02

Create the testbench tb_post_randomize.sv:

- Include the ip_packet.sv file.
- In an initial block, create and randomize an instance of the ip_packet class.
- Call the display method.

Automate the check: After displaying the packet, call the verify_checksum() method using an assert statement (e.g., `assert(pkt.verify_checksum());`). This will make the testbench automatically fail if the checksum logic is incorrect.